

CHƯƠNG 1

1. What is software process, software process activities?

Process:

- A series of steps
- Pre-defined, well-organized
- Produce software product

Main phases:

- Requirement Analysis
- Architecture & Design
- Code implementation
- Testing
- Maintenance

Requirement Analysis:

- Answer question: WHAT
- Identify what to do.
- Role: BA (Business Analyst).
- Activities:
 - o Gather user needs.
 - o Write software requirement specifications.
 - o Verify requirements.
 - o Model requirements.

Architecture & Design:

- Answer question: HOW.
- Find solutions and how to do.
- Role: software architect.
- Activities:
 - o High-level design: Architecture design.
 - o Low-level design: Data, UI, Interaction design.

Code implementation:

- Produce code from design.
- Role: developer
- Activities:
 - o Coding/ Debugging.
 - o Implement database

Testing:

- Find errors from code:
 - o Based on design & requirements
 - o Based on users.
- Role: tester
- Activities: Unit, Integration, System, Acceptance test.

Maintenance:

- Operate, update, extend
- Role: Technical support team, developer.
- Activities:
 - o Set up computers & network
 - o Install & set up system
 - o Report & fix bugs

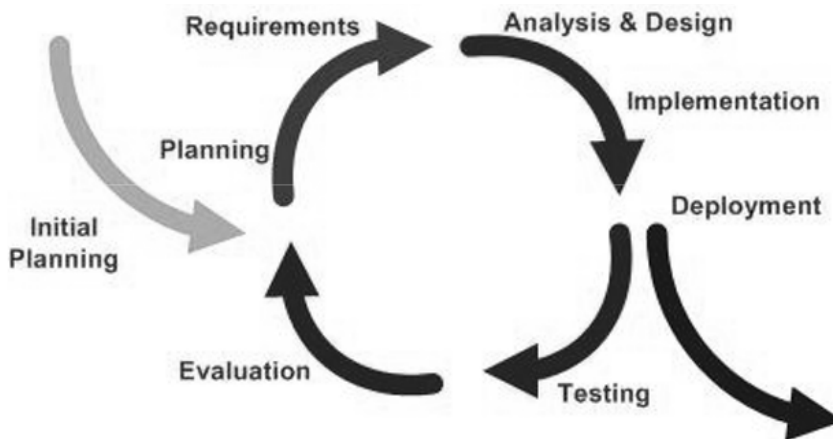
CHƯƠNG 2

1. Phân biệt các quá trình – tính chất (có thể hỏi TN)

	Waterfall	Evolutionary
Characteristics	- SEQUENTIAL and INDEPENDENT phases. - Drafting before implementation. - Testing after implementation	- Improve Code & Fix - ITERATIVE phases. - Fast draft and implementation. - Improve through versions and feedback
Advantages	- Pre-defined and well-organized. - Planning. - Predictive process.	- Requirements can be changed and unclear. - Adaptive process
Disadvantages	- Not welcome changes - Suitable for well-defined and stable requirements. → Improve: Backward Waterfall	- Unpredictable - Low quality design

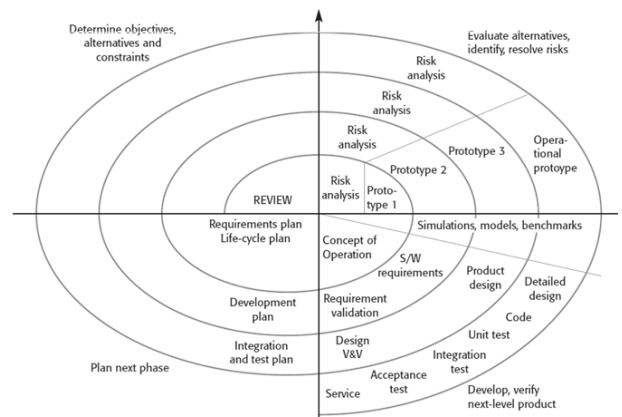
Used Evolutionary Models:

- Iterative Waterfall:

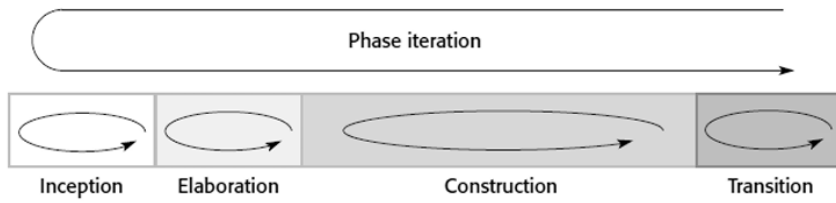


- Spiral: Each phase is a spiral:

- o Objective identification
- o Risk evaluation.
- o Implementation
- o Plan next phase



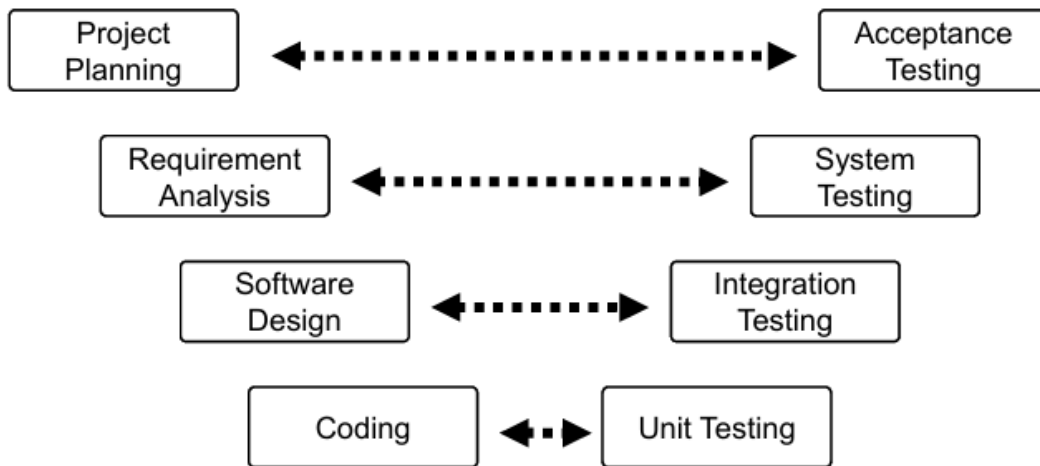
- **RUP (Rational Unified Process)**



- **Four phases:**

- Inception: planning
- Elaboration: analysis & design
- Construction: implementation & testing.
- Transition: maintenance

2. V-Model:



Unit testing:

- Verify small units in software (Function)
- Test each unit in isolation
- Developer
- Document: unit test case

Integration Testing:

- Verify group of units in software.
- Test collaboration of programming functions.
- Developer
- Document: integration test case.

System Testing:

- Verify whole system in operation.
- Test system meets user **requirements**.
- Operate in simulation environment.
- Types:
 - o Functional Testing
 - o Non-functional Testing.

Acceptance Testing:

- User testing
- Test system meets user expectations
- Operate in real-time environment.
- Types: Alpha, Beta Testing

CHƯƠNG 3

1. Vai trò PM là gì, các hoạt động mà ng quản lí dự án cần làm gì. xử lí tình huống: nếu ng muốn rời khỏi team, KH thay đổi ý định, xung đột trong nhóm. 90% cho thi đề xuất lên kế hoạch cho dự án đc đưa

Role: Guarantee successful project (Scope, Budget, Time,...)

Initiation:

- Preliminary study and survey.
- Goals: Feasibility study, Scope identification, Time & budget draft
- Deliverables: Feasibility Report, Project Proposal
- Proposal:
 - o Description (overview, difficulty)
 - o Solutions:
 - Software: features, activity models
 - Hardware: computers, network
 - Business: changed process
 - o Milestones: Software process phases, deliverables
 - o HR & budget (role, Effort & cost estimations)

Planning:

- Work schedule: Who, When, What
- Risk Plan:
 - o Identify risks
 - o Risk analysis: probability, scope of effect, what to do
- Workspace (For documents, codes)
- Communication (email, forum, group)
- Rules (central or remote, meeting & reports, solving conflicts)

Executing:

- Master plan
- Weekly plan (tasks, assigned members, expected results)

Tracking:

- Guarantee plans (Where, what, why)
 - Change Management (Track changes, Scope of effect, keep 3 factors, give buffers)
 - Weekly, biweekly reports
 - Timesheet
2. Success criteria?

CHƯƠNG 4

1. Phân biệt non-functional & functional requirements cho ví dụ

Aspect	Functional Requirements (FRs)	Non-Functional Requirements (NFRs)
Definition	- Users can use. - Interactions between users and system. → Features	No interactions → Constraints on software
Focus	System behavior and functions	Quality constraints. Business constraints Environment constraints
Examples	- User login with username & password - Send confirmation email - Upload/download files	- Response time < 2s - 99.9% uptime - Handle 10,000 users
Measurement	Verified via use-case tests	Verified via metrics & benchmarks (speed, reliability)
Importance	Ensures correct features for users	Ensures usability, efficiency, and reliability
Relation to users	Directly satisfies business/user needs	Ensures a good user experience

2. Mqh giữa non-functional & functional requirements với kiến trúc hệ thống là gì? Cho 3 ví dụ (liên quan tới bài 6)

Functional: Directly influence the logical structure and modules of the architecture.

Non-functional: Define the quality constraints for the architecture.

System Architecture: Implementation of both FRs and NFRs.

Without addressing NFRs, architecture may deliver correct functions but fail in **real-world usage** (e.g., too slow, insecure, or unreliable).

3. 3 essential non functional requirements for cái gì đó (vd ứng dụng phần mềm như coursera) explain why each has been priority

Performance (how fast response) Speed & responsiveness for good UX

Security (authentication, authorization, encryption, audit logging) Data protection & compliance.

Reliability & Availability (uptime, fault tolerance, disaster recovery) Consistent uptime & fault tolerance

CHƯƠNG 5:

1. Cho mô tả vẽ bảng use case (gồm các mục: **usecaseID**, **usecase Name**, **brief description**, **actors**, **basic flow**, **alternative flows**, **special requirements**, **pre-conditions**, **post-conditions**, **extension points**) → tự nhớ bảng, chứ đề ko nói

CHƯƠNG 6:

1. Học mấy cái architecture pattern (hỏi TN), khi nào dùng nó

2. Mqh giữa non-functional requirements vs architecture (cho 3 vd ngoài)

CHƯƠNG 7: khum có

CHƯƠNG 8:

1. UI design principles

Color:

- Consistency
- Simply
- Do not use too many colors
- Be careful of contrast colors

Message:

- Consistency.
- Politeness.
- Simply.
- Informative.
- Informative.
- Use user language (general, special, technical)

Data validation

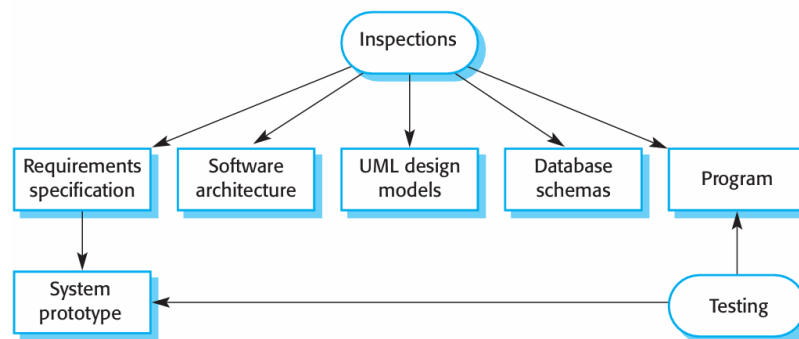
- Do not trust the users → Check all user input data
- Validations constraints (Natural or business)
- Validation (Early or late checking)

2. Đọc mô tả r thiết kế màn hình trên thiết bị gì đó (1 hoặc 2 màn hình), giải thích tại sao lại vẽ như vậy (trong CW8) có vẽ. Phương pháp là:

- Xem coi 1 chức năng có thể đc chia ra **bao nhiêu màn hình phù hợp**
- Bao nhiêu lần thao tác** (click chuột,...), tổng hợp các thao tác đó thành min nhất → chọn màn hình phù hợp

CHƯƠNG 9:

- Thi có vẽ 4 test case cho 1 funct (gồm các mục: **testcaseID, testcase Name, description, relate usecase, input data, expected output, test steps**)
- Inspections and testing + vd



Aspect	Inspections	Testing
Nature	Static (no execution)	Dynamic (requires execution)
Target	Documents, code, design	Running software system
Goal	Detect defects early	Verify correctness, performance
Timing	Early in SDLC	After implementation
Example	Code review, requirement walkthrough	Unit test, system test, load test

3. Mô hình test (slide 14)

4. Nêu quy trình test (slide 15), tại sao pha đó quan trọng trong quá trình (có thể là hỏi pha validation quan trọng trong quá trình phát triển phần mềm ntn)

Criteria	Verification	Validation
Purpose	Does software meet user requirements?	Does software meet user expectations??
Approach	Find defects	Find misunderstandings
Role	Development team	Stakeholders
Methods	Operate/Not operate software	Must operate software
Question	Build the thing right??	Build the right thing??

Verification:

Dynamic

- Verify by operating software (Blackbox)
- Based on Input/Output
- Advantages: low-cost for general, verify non-functional requirements
- Disadvantages: High-cost for specific, software must be finished, hidden bugs

Static

- Verify by review & proofs (white-box)
- Based on internal contents
- Advantages: Software need not be finished, need not to operate, reveal hidden bugs
- Disadvantages: high-cost, review expert, time

Validation:

- Alpha Testing: Limited release, selected users, controlled operation
- Beta Testing: Unlimited release, public users, free operation

5. Test plan

CHƯƠNG 10:

1. So sánh **plan-driven & agile**. Trình bày pp, cho vd, khi nào dùng

Plan-driven development

- A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
- Not necessarily waterfall model – plan-driven, incremental development is possible
- Iteration occurs within activities.

Agile development

- Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

2.

	Waterfall	Evolutionary
Characteristics	- SEQUENTIAL and INDEPENDENT phases. - Drafting before implementation. - Testing after implementation	- Iterative & incremental cycles (sprints) - Working software delivered frequently - Welcomes changing requirements - Continuous collaboration
Advantages	- Pre-defined and well-organized. - Planning. - Predictive process.	- High flexibility & adaptability - Early delivery of value - Encourages customer feedback - Better risk management through short iterations - Higher customer satisfaction
Disadvantages	- Not welcome changes - Suitable for well-defined and stable requirements. → Improve: Backward Waterfall	- Harder to predict timeline & cost - Can lead to scope creep if not managed - Requires high customer involvement - Documentation may be weaker - Less suited for regulated environments

3. Đặc điểm : agile methods, manifesto, 5 nguyên tắc của agile → vận dụng

Feedback.

Simplicity.

Communication.

Welcome Changes

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

4. Đặc điểm của scum

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2-4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

38

Defect testing (unit, component, system)

Validation testing (user & release testing)